

COMP 2233 Data Structures Course Term Project Report

- **Project Name:** Fast Learning Management with Bloom Filter
- **Student Name&Surname:** Arda Durmaz
- **Student Number:** 23070001009
- **Project Advisor:** Assoc. Prof. Ömer ÇETİN
- **Date of Submission:** 27.12.2025

Ethical Statement

I acknowledge and declare that I have personally worked in accordance with academic ethical principles during the preparation of this project. I accept and undertake that if the contrary situation is detected in any way, my project work will not be accepted and will not be evaluated, and I will be subject to sanctions announced by the university administration.

Student:

Signature:

Arda Durmaz

23070001009

27.12.2025

Given Term Project:

28. Fast Learning Management with Bloom Filter

- **Description:** Develop a solution using the Bloom Filter data structure to quickly check if elements are present in a large data set. This data structure will allow for fast searching without guaranteeing accuracy.
- **Data Structure :** Bloom Filter

- **Expected Output:** A fast system that probabilistically checks if a given element is present in a data set.

1.Introduction

This project aims to implement a system that checks the existence of certain data in a larger data set using Bloom Filter data structure. In this project, we are only interested in whether the data exists or not. Thus, Bloom Filter is an efficient and suitable algorithm for this purpose.

2.Project Purpose

The main purpose of this project is to quickly and efficiently check the membership of an item in a data set. Since we are not interested in what the data really is, we don't have to store the exact data, rather than store its mapped version in a Boolean array using various hash functions. This way we save up a lot of memory space.

3.Data Structures Used

3.1 Bloom Filter

Bloom Filter was used in this project to learn whether an item is in a set with a probability. It cannot verify that the item is in a set for sure (false positives), but it ensures that the item is *not* in the set. This false positive possibility can be reduced by choosing the optimal number of hash functions k given by the formula $k = \ln 2 (m/n)$ where m is the size of the Boolean array and n is the number of items represented in Boolean array as true.

4.System Design

The program uses Bloom Filter to map the items into an initially falsified Boolean array as true or false binary representation. Before mapping the item, it is processed through several hash functions which produce different numbers. These numbers might be bigger than the size of the Boolean array, thus we use mod operators to keep the numbers in range

of $0-l$ where l is the size of array. After obtaining the numbers, we change those positions -which align with numbers- to true. This way we keep track of which item has been used.

5.Algorithm

The algorithm uses 5 hash functions which are *djb2*, *jenkins one at a time* and *murmurhash3* with 2 more different seeds. A Boolean array is initialized at the beginning of the algorithm as all its values falsified. Then it runs the hash functions and records the values in array. Also, there is a function that checks the membership of an item.

6.C++ Code

In this project C++ is used because it possesses helpful built-in functions with `<string>`, `<cstdint>`, `<cstring>` libraries.

Here is the C++ code of the program :

```
#include <string>
#include <cstdint>
#include <cstring>

using namespace std;

const int MAX_SIZE = 7500;

bool bit_array[MAX_SIZE];

void init_bitArray(){
    for(int i = 0; i < MAX_SIZE; i++){
        bit_array[i] = false;
    }
}

uint32_t djb2(string data) {
    uint32_t hash = 5381;
    for (char c : data) {
        hash = ((hash << 5) + hash) + (uint8_t)c;
    }
    return hash;
}

uint32_t jenkins_one_at_a_time(string data) {
    uint32_t hash = 0;
    for (char c : data) {
        hash += (uint8_t)c;
        hash += (hash << 10);
    }
}
```

```

    hash ^= (hash >> 6);
}
hash += (hash << 3);
hash ^= (hash >> 11);
hash += (hash << 15);
return hash;
}

static inline uint32_t murmur_32_scramble(uint32_t k) {
    k *= 0xcc9e2d51;
    k = (k << 15) | (k >> 17);
    k *= 0x1b873593;
    return k;
}

uint32_t murmur_hash3(const uint8_t* key, size_t len, uint32_t seed) {
    uint32_t h = seed;
    uint32_t k;
    for (size_t i = len >> 2; i--;) {
        memcpy(&k, key, sizeof(uint32_t));
        key += sizeof(uint32_t);
        h ^= murmur_32_scramble(k);
        h = (h << 13) | (h >> 19);
        h = h * 5 + 0xe6546b64;
    }
    k = 0;
    for (size_t i = len & 3; i--;) {
        k <<= 8;
        k |= key[i - 1];
    }
    h ^= murmur_32_scramble(k);
    h ^= len;
    h ^= h >> 16;
    h *= 0x85ebca6b;
    h ^= h >> 13;
    h *= 0xc2b2ae35;
    h ^= h >> 16;
    return h;
}

uint32_t hash_string(const std::string& str, uint32_t seed) {
    return murmur_hash3(reinterpret_cast<const uint8_t*>(str.data()), str.size(), seed);
}

void record_to_filter(string data){
    bit_array[djb2(data) % MAX_SIZE] = true;
    bit_array[jenkins_one_at_a_time(data) % MAX_SIZE] = true;
    bit_array[hash_string(data,1) % MAX_SIZE] = true;
    bit_array[hash_string(data,2) % MAX_SIZE] = true;
    bit_array[hash_string(data,3) % MAX_SIZE] = true;
}

bool check_data(string data){

    if(bit_array[djb2(data) % MAX_SIZE] == false){
        return false;
    }
    if(bit_array[jenkins_one_at_a_time(data) % MAX_SIZE] == false){
        return false;
    }
    if(bit_array[hash_string(data,1) % MAX_SIZE] == false){
        return false;
    }
    if(bit_array[hash_string(data,2) % MAX_SIZE] == false){
        return false;
    }
    if(bit_array[hash_string(data,3) % MAX_SIZE] == false){

```

```
    return false;
}
    return true;
}
```

7.Evaluation

Bloom Filter data structure is very efficient for checking the existence of an item. In addition, it saves too much memory because it doesn't use the exact data, but the mapped version of the data that costs much less memory. However, there is a flaw in the program where false positives might occur. We use this algorithm knowing that we trade memory space with precision.

7.1 Complexity

Adding and checking with bloom filters is constant time, $O(k)$ where k is the number of hash functions used.

The space complexity of the algorithm is $O(m)$ where m is the length of the Boolean array.

8.Resources

- [MurmurHash - Wikipedia](#)
- [Bloom filter - Wikipedia](#)
- [Jenkins hash function - Wikipedia](#)

